

University of The Gambia
School of Information & Communication Technology

Design & Architecture Document

A Multi-Paradigm, Enterprise-Ready University Course Registration System

Course CPS 314 — Programming Language Concepts
Assignment Three-Week Practical (Multi-Paradigm Enterprise System)
Authors Ousman Bah (22423253)
Kiera J. J. Jackson (22326026)
Languages Java (Spring Boot) + Python (FastAPI)
Repository github.com/ob22423253-web/course-registration-system

1. Introduction & System Overview

This document describes the design of a University Course Registration System built to demonstrate the core concepts of CPS 314: paradigm selection, abstract data types, generics, concurrency, exception handling, formal syntax, and meaningful integration between two languages drawn from different paradigms. The system is split into two cooperating services that communicate over a REST contract.

The **object-oriented module** is written in Java with Spring Boot. It owns the domain (students, courses, enrollments), persists state to MongoDB Atlas, enforces business rules, and handles concurrent registration safely. The **functional module** is written in Python with FastAPI. It performs analytical and rule-based computation — GPA calculation, student ranking, prerequisite-chain resolution, and evaluation of a small prerequisite rule language. When the Java service must decide whether a student is eligible for a course, it calls the Python service and uses the functional result to drive the outcome.

This division is deliberate. Each language is used where its paradigm is strongest: Java for stateful, concurrent, type-safe orchestration; Python for concise, expression-oriented, immutable computation. The remainder of this document justifies that choice formally and maps every required language feature to the place in the codebase where it is implemented.

1.1 High-Level Architecture

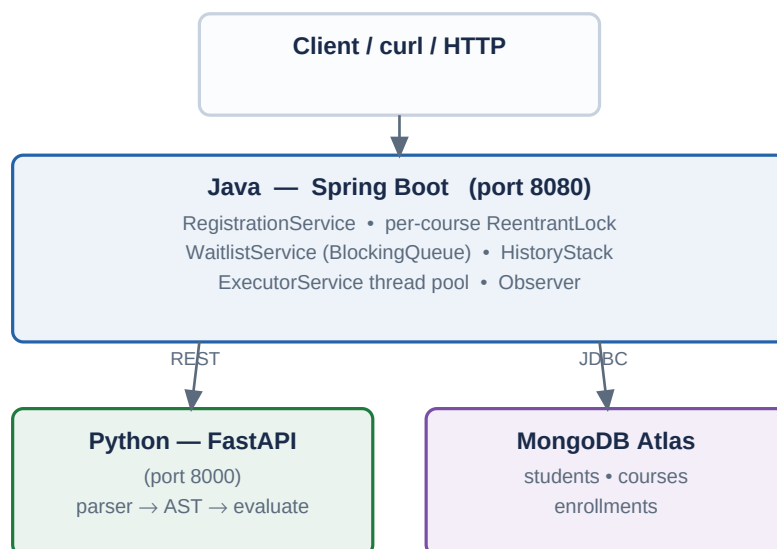


Figure 1 — Two services over a REST contract, with MongoDB persistence. The Java service is the single writer; Python is a stateless compute service.

The flow of a registration request is: the client posts to the Java service; Java loads the student and course from MongoDB; Java sends the student's status, GPA, completed courses, and the course's prerequisite rule to Python; Python parses and evaluates the rule functionally and returns an eligibility verdict; Java then enrolls the student, places them on the waitlist if the course is full, or rejects with a human-readable reason. If Python is unreachable, Java *fails closed* (denies rather than wrongly enrolling) — a deliberate safety decision.

2. Choice of Languages & Justification

The assignment requires two languages from different paradigms, justified on four grounds: language evaluation criteria, influences on language design, paradigm suitability, and the compilation/runtime model. Each is addressed below for Java and Python.

2.1 Language Evaluation Criteria

Using the classic criteria of readability, writability, reliability, and cost:

Criterion	Java	Python
Readability	Verbose but explicit; static types document intent. Strong, tooling	More succinct, functional syntax, expression-oriented, close to pseudocode
Writability	More ceremony, but generics and IDE support speed large systems	Speed large systems, lambdas, comprehensions, pattern matching.
Reliability	Static typing + checked exceptions catch errors at compile time.	Dynamic typing defers errors to runtime; mitigated by tests.
Cost	Higher development effort; excellent long-term maintainability	Lower development cost; fast to prototype analytical logic.

2.2 Influences on Language Design

- **Java** inherits its C/C++-style syntax and object model, but replaced manual memory management with garbage collection and pointers with references. Its defining design goal — *write once, run anywhere* — produced the JVM, making portability a first-class concern.
- **Python** descends from the ABC and Modula-3 traditions, prioritising readability and rapid development. Significant indentation, dynamic typing, and a large standard library reflect a design philosophy that favours programmer productivity over raw performance.

2.3 Paradigm Suitability

Java is an object-oriented, statically typed language well suited to the parts of this system that are stateful and concurrent: encapsulated domain objects, a registration service guarded by locks, a thread pool, and a producer–consumer waitlist. Its type system and checked exceptions make these safety-critical paths reliable. Python is multi-paradigm with strong functional support, making it the natural home for the analytical module: pure functions, immutable data, higher-order pipelines, recursion over prerequisite chains, and pattern matching over rule and status types.

2.4 Compilation & Runtime Model

Java is compiled ahead of time to platform-independent *bytecode*, which the JVM executes and just-in-time compiles to native code at runtime. This yields strong performance and early error detection at the cost of a build step and a heavier runtime. Python is *interpreted* by CPython, which compiles to bytecode transparently and executes it on a stack machine; there is no separate build step and types are checked dynamically at runtime. The trade-off is clear in this project: Java's model suits the long-lived, concurrent backend, while Python's model suits the quick-to-change analytical service.

3. Language Features & Where They Live

Every feature required by the rubric is implemented in working code. The table below maps each concept to its location in the repository so it can be located quickly during the defense.

Concept	Module	Location
Generics (own type)	Java	adt/Repository<T,ID>, adt/Result<T> (sum type, map/flatMap)
Custom ADT 1 — FIFO queue	Java	adt/WaitlistQueue<E> (thread-safe)
Custom ADT 2 — LIFO stack	Java	adt/HistoryStack<E> (registration undo)
Custom ADT 3 — Repository	Java	adt/Repository<T,ID> interface + Mongo adapter
Encapsulation	Java	domain/* private fields, getters/setters
Inheritance + interface	Java	domain/Person → Student; observer abstract → concrete
Exception hierarchy	Java	exception/* (checked base + unchecked branch)
Exception recovery	Java	RegistrationService (CourseFull → waitlist)
Functional interface	Java	functional/RegistrationValidator (@FunctionalInterface)
Lambdas / streams	Java	CourseService (Predicate, Comparator, map/filter/collect)
Concurrency 1 — pool	Java	service/concurrent/RegistrationExecutor
Concurrency 2 — prod/cons	Java	WaitlistService + WaitlistConsumer (BlockingQueue)
Caching	Java	config/CacheConfig (Caffeine on course lookups)
Design pattern — Observer	Java	observer/WaitlistSubject + observers
Dependency injection	Java	Spring @Service / @Component throughout
Lambdas	Python	functional.py, gpa.py
Functional composition	Python	functional.py (compose, pipe)
Higher-order / pipelines	Python	functional.py (filter_map, reduce, generators)
Recursion	Python	prereq.py (required_chain – transitive resolver)
Immutable constructs	Python	@dataclass(frozen=True) AST nodes, frozenset
Pattern matching	Python	eligibility.py, parser.py (match/case)
EBNF parser	Python	parser.py (recursive-descent, AND/OR precedence)
REST integration	Both	Java PythonAnalyticsClient → FastAPI /api/v1/eligibility

4. Formal Syntax: The Prerequisite Rule Language

The system needs a small domain-specific language so administrators can express course prerequisites declaratively, for example:

```
RULE CS201 REQUIRES CS101 AND MTH110
RULE CS301 REQUIRES CS201 OR CS250
RULE CS401 REQUIRES CS301 AND (CS250 OR CS260)
```

These rules are not merely documented on paper — they are parsed and evaluated by the Python module at runtime (`parser.py`), connecting the formal grammar directly to working code.

4.1 EBNF Grammar

EBNF is chosen over pure BNF because its repetition operator `{ }` expresses variable-length course codes (`CS101`, `MTH110`) and chained operators without bloating the grammar.

```
rule = "RULE" , course , "REQUIRES" , expression ;
expression = term , { "OR" , term } ;
term = factor , { "AND" , factor } ;
factor = course | "(" , expression , ")" ;
course = letter , letter , { letter } , digit , digit , digit ;
letter = "A" | "B" | ... | "Z" ;
digit = "0" | "1" | ... | "9" ;
```

The grammar is layered so that `AND` binds tighter than `OR`: a *term* (`AND`-level) sits inside an *expression* (`OR`-level), and a *factor* is the smallest unit — a single course code or a parenthesised sub-expression. This layering is what resolves operator precedence, discussed in 4.3.

4.2 Parse Tree for a Sample Statement

For the statement **RULE CS401 REQUIRES CS301 AND (CS250 OR CS260)**, the parser produces the tree below.

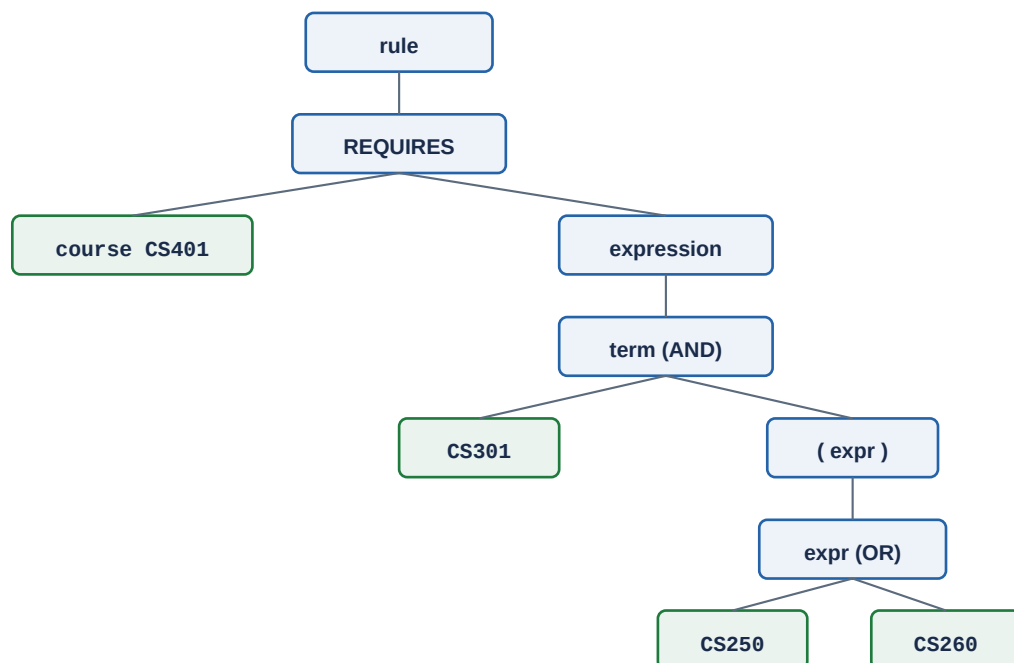


Figure 2 — Parse tree. Green leaves are terminal course codes; blue nodes are grammar non-terminals. The parentheses force `CS250 OR CS260` to group beneath the outer `AND`.

4.3 Ambiguity & Its Resolution

Consider the rule **CS201 OR CS250 AND CS101** with no precedence convention. It has two valid readings:

- (CS201 OR CS250) AND CS101 — requires CS101 plus one of the first two.
- CS201 OR (CS250 AND CS101) — requires CS201 alone, or both of the others.

These mean different things for whether a student may register, so an ambiguous grammar is unacceptable in a registration system. The ambiguity is resolved **structurally**: because the grammar nests *term* (AND) inside *expression* (OR), AND is given higher precedence and the language produces only the second reading. Parentheses let an administrator override the default when the first reading is intended. This mirrors arithmetic precedence ($\times$ before $+$), which makes the behaviour intuitive. The recursive-descent parser in `parser.py` implements exactly this nesting, so the resolved grammar and the running code agree by construction.

5. UML Class Diagram (Java Module)

The diagram summarises the principal Java classes, the ADTs, the exception hierarchy, and the Observer collaboration. Generics are shown with angle brackets; the dashed arrow marks the REST call into the Python service.

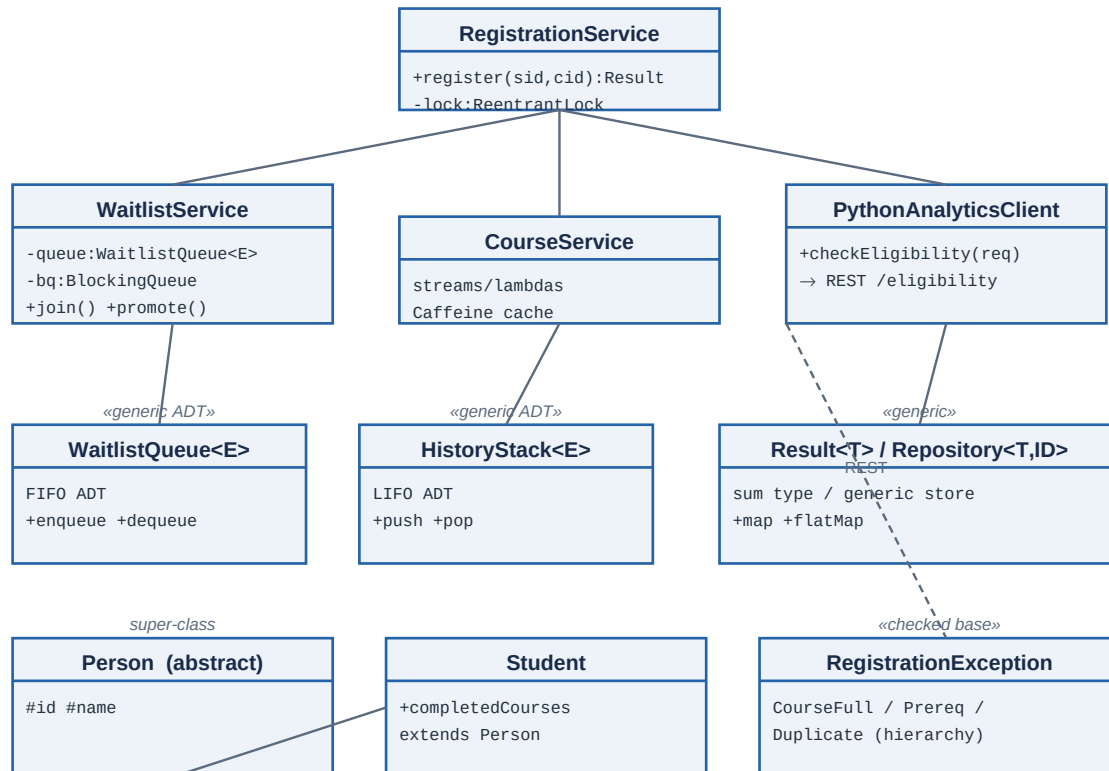


Figure 3 — Principal classes of the Java module. Generic ADTs are stereotyped `«generic»`; **Student** inherits from the abstract **Person**; the exception hierarchy roots at a checked **RegistrationException**.

5.1 Concurrency & Safety Summary

Two concurrency mechanisms are implemented. First, an `ExecutorService` thread pool processes many registration requests in parallel. Second, a producer–consumer waitlist uses a `BlockingQueue`: full-course registrations are produced onto the waitlist, and a daemon consumer thread promotes students when seats free up, firing an `Observer` notification. A per-course `ReentrantLock` together with an optimistic version field guarantees that capacity is never exceeded. A stress test of 200 concurrent requests against 30 seats confirms the invariant: exactly 30 enrolled, 170 waitlisted, 0 lost, 0 overbooked.

5.2 Testing Summary

The system is covered by 51 automated tests: 22 JUnit 5 tests over the ADTs, exception hierarchy, validators and registration flow; 25 pytest tests over the parser, recursion, pattern matching and functional pipeline; a Java↔Python integration round-trip; an error-recovery test (course-full → waitlist); and the concurrency stress test described above. All tests pass.

5.3 Benchmark Interpretation

The stress test drives 200 concurrent registration requests at a single course with 30 seats and reports, for one run: concurrent throughput of about 217 requests/second against a sequential baseline of about 270 requests/second, with average latency near 4.6 ms. Two observations matter. First, **correctness held under contention**: exactly 30 students enrolled, 170 were waitlisted, none were lost and none overbooked — the per-course ReentrantLock and the optimistic version field did their job. Second, the concurrent run is *slightly slower* than the sequential one, which is expected rather than alarming. Because every request contends for the same course's lock, the seat-changing critical section is effectively serialised; the threads add scheduling and lock-handoff overhead without being able to parallelise the one resource they all fight over. This is the classic trade-off of lock-based synchronisation: it buys safety at the cost of throughput on a hot, single-resource path. In production the contention would be spread across many courses, so real concurrency would recover the throughput while preserving the same safety guarantee.

Both services are deployed as Docker containers on Render.com and persist to MongoDB Atlas, demonstrating the system runs as a real, integrated, multi-paradigm application end to end.